

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. G. A. Senthil

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Program:

```
import gymnasium as gym
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
env = gym.make("CartPole-v1")
state, info = env.reset()
```

```
action = env.action_space.sample()
state, reward, terminated, truncated, info = env.step(action)
```

```
print("State:", state)
print("Reward:", reward)
```

```
env.close()
```

Sample input:
Environment : CartPole-v1
Action : Random (0 or 1)

Sample Output

Initial State: [0.012 -0.018 0.035 0.041]

Action Taken: 1

Next State: [0.011 0.174 0.036 -0.245]

Reward: 1.0

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)

state, info = env.reset()

print("Initial State:", state)

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)

    print("Step:", i+1)
    print("Action:", action)
    print("State:", state)
    print("Reward:", reward)

    if terminated or truncated:
        break
```

```
env.close()
```

Sample Input

Environment : FrozenLake-v1
Actions : Random

Sample Output

Initial State: 0

Step: 1

Action: 2
State: 1
Reward: 0.0

Step: 2
Action: 1
State: 5
Reward: 0.0

Step: 3
Action: 2
State: 6
Reward: 0.0

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Program:

```
import gymnasium as gym
```

```
env = gym.make("CartPole-v1")
```

```
state, info = env.reset()
```

```
total_reward = 0
```

```
for step in range(20):
```

```
    action = env.action_space.sample()
```

```
    state, reward, terminated, truncated, info = env.step(action)
```

```
    total_reward += reward
```

```
    print("Step:", step+1)
```

```
    print("Action:", action)
```

```
    print("Reward:", reward)
```

```
    if terminated or truncated:
```

```
        break
```

```
print("Total Reward:", total_reward)
```

```
env.close()
```

Sample Input

Environment : CartPole-v1

Policy : Random

Sample Output

Step: 1

Action: 0
Reward: 1.0

Step: 2
Action: 1
Reward: 1.0

Step: 3
Action: 1
Reward: 1.0

Total Reward: 20.0

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Program:

```
states = ["A","B","C"]
```

```
transitions = {  
    "A":("B",5),  
    "B":("C",10),  
    "C":("C",0)  
}
```

```
state="A"
```

```
total=0
```

```
for i in range(3):  
    next_state,reward=transitions[state]  
  
    print(state,"->",next_state,"Reward:",reward)
```

```
    total+=reward
```

```
    state=next_state
```

```
print("Total Reward:",total)
```

Sample Input

Initial State : A

Sample Output

A -> B Reward: 5

B -> C Reward: 10

C -> C Reward: 0

Total Reward: 15

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Program:

```
gamma=0.9
```

```
reward=10
```

```
next_value=5
```

```
value=reward+gamma*next_value
```

```
print("Reward:",reward)
```

```
print("Discount:",gamma)
```

```
print("State Value:",value)
```

Sample Input

Reward = 10

Discount Factor = 0.9

Next State Value = 5

Sample Output

Reward: 10

Discount: 0.9

State Value: 14.5

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Program:

```
import numpy as np
```

```
import random
```

```
values=[5,2,8]
```

```
epsilon=0.2
```

```
if random.random()<epsilon:  
    action=random.randint(0,2)
```

```
else:  
    action=np.argmax(values)
```

```
print("Selected Action:",action)
```

Sample Input

Values = [5,2,8]

Epsilon = 0.2

Sample Output

Selected Action: 2

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Program:

```
import numpy as np
```

```
rewards=[1,4,8]
```

```
counts=np.zeros(3)
```

```
values=np.zeros(3)
```

```
for i in range(20):
```

```
    arm=np.argmax(values)
```

```
    reward=rewards[arm]
```

```
    counts[arm]+=1
```

```
    values[arm]+= (reward-values[arm])/counts[arm]
```

```
print(values)
```

```
print("Best Arm:",np.argmax(values))
```

Sample Input

Rewards = [1,4,8]

Sample Output

Estimated Rewards:

[1.0 4.0 8.0]

Best Arm: 2

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Program:

```
import matplotlib.pyplot as plt
```

```
reward=[2,3,5,6,8,9]

plt.plot(reward)

plt.xlabel("Episodes")

plt.ylabel("Reward")

plt.title("Reward Graph")

plt.show()
```

Sample Input

Reward = [2,3,5,6,8,9]

Sample Output

Graph Displayed

X-axis : Episodes

Y-axis : Reward

Increasing reward curve

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Program:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model=Sequential()
```

```
model.add(Dense(16,input_shape=(4,),activation="relu"))
```

```
model.add(Dense(16,activation="relu"))
```

```
model.add(Dense(2))
```

```
model.summary()
```

Sample Input

Input Shape = (4,)

Hidden Neurons = 16

Output = 2

Sample Output

Layer (type)

Dense

Dense

Dense

Total Parameters: 370

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Program:

```
import gymnasium as gym
```

```
env=gym.make("CartPole-v1")
```

```
state,info=env.reset()
```

```
reward_total=0
```

```
for i in range(20):
```

```
    action=env.action_space.sample()
```

```
    state,reward,terminated,truncated,info=env.step(action)
```

```
    reward_total+=reward
```

```
    if terminated or truncated:
```

```
        break
```

```
print("Episode Reward:",reward_total)
```

```
env.close()
```

Sample Input

Environment = CartPole-v1

Policy = Random

Sample Output

Episode Reward: 20.0

Code of Conduct:

*I **Kandukuri.Kavya-192425283**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kandukuri.Kavya
192425283

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

